

VIT BHOPAL UNIVERSITY

VITyarthi – Your Learning Destination

PROJECT INFORMATION

Project title : Bank management system

Course title : Introduction To Problem Solving And programming

Course code : CSE 1021

Course type : Flipped course

Course credits : 4

Professor : Dr. Manoj Kumar

Slot : B11+B12+B13+C14+E11+E12

Submitted by : Harsh Patil

Registration number : 26BAI10813

College : VIT Bhopal University

Program : B.Tech CSE AI/ML

Year / Semester : 1st Year, 1st Semester

Submission date : 30/09/26

INTRODUCTION :

Banks manage very large numbers of customer accounts, and every one of them depends on a small set of operations being carried out reliably: opening an account, depositing money, withdrawing money and checking a balance. If any of these operations accepts wrong data, such as a negative deposit or a withdrawal larger than the balance, the records stop being trustworthy. This is why even the simplest banking software is built around careful validation.

This project is a small-scale simulation of such a system, written in Python with a menu-driven console interface. A user chooses an option from a numbered menu, supplies the required details, and the program validates the input, updates the account data and reports the result. Control then returns to the menu so that another operation can be performed.

The application is intentionally lightweight. It uses only core Python features (functions, dictionaries, loops and conditional statements) and needs no external library or framework, which keeps it easy to install, run, review and grade as a first-year academic project.

Key characteristics of the system

- Menu-driven interaction
- Dictionary-based account storage
- Input validation on every operation
- Function-per-operation code structure
- Clear error messages for invalid actions

PROBLEM STATEMENT :

Manual record keeping of customer accounts, using registers or loose spreadsheets, is slow and error-prone. Balances can be mis-calculated, the same account number can be issued twice, and overdrawn withdrawals may go unnoticed. A small structured program can remove these problems by keeping all accounts in one place and enforcing the rules automatically every time a transaction is attempted.

The proposed system accepts an account number, a holder name and an opening balance to create an account. It then supports balance enquiry, deposits, withdrawals and a listing of all accounts, rejecting any request that breaks the defined rules.

The project focuses on a clearly defined academic scope rather than attempting to reproduce a full commercial banking platform. The goal is correctness and clarity of implementation rather than breadth of features.

PROJECT OBJECTIVES :

The main objective is to develop an understandable and reliable Python program that models basic banking operations. Additional objectives are to apply programming concepts in a complete project workflow, from requirements through design, implementation and testing.

- Apply core Python concepts: functions, dictionaries, loops and conditionals.
- Create accounts with a unique account number, holder name and opening balance.
- Model deposits, withdrawals and balance enquiries with proper validation.
- Prevent duplicate accounts and overdrawn withdrawals.
- Display all stored accounts in a readable form.
- Provide a simple, user-friendly menu interface that never crashes on an invalid menu choice.
- Test the program with a documented sequence of inputs and record the results.

Together, these objectives ensure that the finished project is not just a working script, but a small, well-organised piece of software that follows recognisable engineering practice.

SCOPE OF THE PROJECT :

The current project covers a single-user, single-session banking simulation through a command-line interface. It focuses on creating accounts and performing the four core operations on them, using only the Python standard language features for storage and computation.

Authentication, permanent storage, interest calculation, transfers between accounts and graphical interfaces are outside the current scope. These are listed under future enhancements, since they would need additional infrastructure beyond what a single-semester course project is expected to deliver.

- In scope: account creation, deposit, withdrawal, balance check, account listing
- In scope: terminal interface
- In scope: input validation
- Out of scope: persistent storage (files or database)
- Out of scope: PIN / password authentication
- Out of scope: GUI or web interface

FUNCTIONAL REQUIREMENTS :

The functional requirements define the operations available to the user through the terminal menu.

- Create a new account with a unique account number, name and opening balance.
- Reject an account number that is already in use.
- Show the holder name and balance of an existing account.
- Deposit a positive amount into an existing account.
- Withdraw a positive amount that does not exceed the current balance.
- Display all accounts, or report that none exist.
- Report "Account not found." for unknown account numbers.
- Handle invalid menu choices without terminating the program.
- Exit cleanly with a thank-you message.

NON-FUNCTIONAL REQUIREMENTS :

Non-functional requirements describe qualities expected from the system, beyond the specific operations it performs.

- Usability: simple numbered menu and clear prompts.
- Reliability: every transaction is validated before it changes a balance.
- Maintainability: one function per operation, so each can be read and changed separately.
- Performance: instant response for the small number of accounts held in memory.
- Resource efficiency: no external libraries required.
- Portability: runs on any system with a standard Python 3 interpreter.
- Testability: operations can be checked repeatedly with a fixed input sequence.

REQUIREMENTS TRACEABILITY MATRIX :

The table below maps each functional requirement to the function primarily responsible for implementing it, and to the test case that verifies it. This traceability view helps confirm that every requirement is both implemented and tested.

Req. ID	Requirement	Responsible function	Verified by
FR-1	Create account with unique number	create_account()	Create account test
FR-2	Reject duplicate account number	create_account()	Duplicate account check
FR-3	Check balance of existing account	check_balance()	Balance enquiry via menu
FR-4	Deposit a positive amount	deposit_money()	Deposit test
FR-5	Withdraw within available balance	withdraw_money()	Withdraw test
FR-6	Reject over-withdrawal	withdraw_money()	Over-withdrawal test
FR-7	Display all accounts	display_accounts()	Display all test
FR-8	Report unknown account	All account operations	Manual run
FR-9	Handle invalid menu choice	Main menu loop	Manual run
FR-10	Exit with thank-you message	Main menu loop	Exit test

SYSTEM ARCHITECTURE :

The application follows a simple layered idea. The main menu loop receives the user's choice and calls the matching operation function. Each function reads from and writes to a single shared data structure, the accounts dictionary, and prints its result back to the console.

Keeping each operation in its own function separates user interaction from business rules to a useful degree. For example, the deposit rule (amount must be greater than zero) lives only inside `deposit_money()`, so it can be changed in one place without touching the menu.

- User → Main menu loop
- Main menu loop → Operation function (create / check / deposit / withdraw / display)
- Operation function → accounts dictionary
- Result message → Console → User

USE CASE ANALYSIS :

The main actor is the Bank User (customer or teller). The user interacts with the system through the terminal menu, and each menu option represents a use case. Every use case begins at the main menu and returns to it once the operation is complete.

- Create Account
- Check Balance
- Deposit Money
- Withdraw Money
- Display All Accounts
- Exit

DETAILED WORKFLOW :

When the program starts, the accounts dictionary is created (it may be pre-filled with sample accounts) and the menu is displayed inside a while True loop. The selected operation is processed and the result is displayed. The loop continues until the user selects the exit option.

For a withdrawal, the system first looks up the account number, then checks that the amount is positive, and finally checks that it does not exceed the balance. Only when all checks pass is the balance reduced. If any check fails, an error message is shown and the balance is left unchanged.

- Start
- Initialise accounts dictionary
- Display menu
- Read choice
- Validate / process request
- Display result
- Return to menu
- Exit on option 6

SEQUENCE OF OPERATIONS :

A typical deposit interaction begins with the user choosing option 3 and entering an account number and amount. The deposit function looks up the account, validates the amount and updates the balance stored in the dictionary.

- User → Menu (choice 3)
- Menu → deposit_money()
- deposit_money() → accounts dictionary (lookup)
- deposit_money() → validation (amount > 0)

VITYARTHI – PROJECT

- deposit_money() → balance updated
- Result → Console → User

MODULE AND COMPONENT DESIGN :

The program is divided into focused functions so that each has a clear, single responsibility. This makes the code easier to navigate, test and extend. The menu is numbered as follows: 1 Create account, 2 Check balance, 3 Deposit, 4 Withdraw, 5 Display all accounts, 6 Exit.

Function	Purpose
create_account()	Adds a new account after checking that the number is not already used.
check_balance()	Shows holder name and balance for an existing account.
deposit_money()	Adds a positive amount to the account balance.
withdraw_money()	Deducts an amount if it is positive and not greater than the balance.
display_accounts()	Lists every account, or reports that none exist.
Main menu loop	Displays the menu, reads the choice and calls the matching function.

Each function has a small, clear purpose, which keeps the code short and makes it straightforward to locate the logic responsible for any menu option.

DATA MODEL AND STORAGE DESIGN :

All accounts are held in a global dictionary named `accounts`. The account number is the key, and the value is another dictionary holding the holder's name and balance. A dictionary was chosen because it gives direct lookup by account number and makes duplicate detection a single membership test.

The data lives only in memory while the program runs. A sample of the structure is shown below.

```
accounts = {  
    "101": {"name": "Amit", "balance": 5000.0},  
    "102": {"name": "Riya", "balance": 12000.0}  
}
```

Because storage is in memory, the data is lost when the program closes. Adding file or database storage is listed under future enhancements.

INPUT VALIDATION AND ERROR HANDLING :

Validation prevents incorrect data from entering the accounts dictionary. Every operation checks its inputs before changing any balance, so a single bad entry cannot corrupt the stored data.

- Duplicate account numbers are rejected.
- Deposit and withdrawal amounts must be greater than zero.
- Withdrawals larger than the balance are rejected (Insufficient balance).
- Unknown account numbers show 'Account not found.'
- Invalid menu choices show an error and the menu is displayed again.

The current version does not use try/except blocks, so non-numeric amount input is not yet handled gracefully. This is recorded honestly as a limitation and planned as a future enhancement.

CORE ALGORITHM LOGIC :

This section describes, at a conceptual level, the logic behind the most important operations. The notation is illustrative pseudocode rather than literal source code.

Create account

```
function create_account():
    read acc_no, name, opening_balance
    if acc_no already in accounts:
        print "Account already exists"
    else:
        accounts[acc_no] = {name, opening_balance}
        print "Account created"
```

Deposit

```
function deposit_money():
    read acc_no, amount
    if acc_no not in accounts: print "Account not found."
    else if amount <= 0:      print "Invalid amount"
    else:
        accounts[acc_no].balance += amount
```

Withdraw

```
function withdraw_money():
    read acc_no, amount
    if acc_no not in accounts: print "Account not found."
    else if amount <= 0:      print "Invalid amount"
    else if amount > balance: print "Insufficient balance"
    else:
        accounts[acc_no].balance -= amount
```

PROCESSING AND BALANCE LOGIC :

Every transaction reads the current balance from the dictionary, applies a rule, and writes the new balance back. A short worked example, using the same sequence as the test table, is shown below.

Step	Operation	Balance before	Balance after
1	Create account 101 (Amit), opening 5000	—	Rs. 5000
2	Deposit 1500	Rs. 5000	Rs. 6500
3	Withdraw 700	Rs. 6500	Rs. 5800
4	Withdraw 99999	Rs. 5800	Rs. 5800 (rejected)

In step 4 the withdrawal is rejected because the amount exceeds the balance. The important point is that the balance stays unchanged: the check happens before any subtraction, which is the core safety rule of the system.

IMPLEMENTATION AND PYTHON CONCEPTS :

The project applies functions, dictionaries, loops, conditional statements, string formatting and console input/output. These concepts work together to create a complete application rather than isolated code examples.

- Dictionaries (nested) for account storage
- Functions and parameters
- while True loop for the menu
- if / elif / else for validation
- Membership testing with the in operator
- Type conversion (float) for amounts
- Console input and formatted output

Using these fundamentals together, rather than in isolation, was a deliberate choice: it mirrors how introductory concepts are combined in real, working software.

TESTING STRATEGY AND TEST CASES :

The program was run with a sample sequence of inputs covering the normal path and one invalid case. Results matched expectations in every case.

Test	Input	Expected result	Status
Create account	101, Amit, 5000	Account created	Pass
Deposit	101, 1500	Balance 6500	Pass
Withdraw	101, 700	Balance 5800	Pass
Over-withdrawal	101, 99999	Insufficient balance	Pass
Display all	Option 5	Amit, Rs. 5800.0	Pass
Exit	Option 6	Thank-you message, program ends	Pass

SAMPLE OUTPUT AND SCREENSHOT PLAN :

The report should use screenshots from the student's own execution. These provide evidence that the implemented functions work as described. The table below lists each screenshot to capture and what it demonstrates.

Screenshot	Purpose
Main menu	Shows all six options at startup.
Creating an account	Shows validated input being accepted.
Deposit and withdrawal	Shows balances changing correctly.
Over-withdrawal	Shows the Insufficient balance message.
Display all accounts	Shows every account and its balance.
Exit	Shows the thank-you message.

[Insert your own runtime screenshots here before submission.]

DESIGN DECISIONS :

The implementation choices were made to keep the project understandable while demonstrating clear organisation.

- One function per operation keeps responsibilities separate.
- A nested dictionary gives direct lookup by account number.
- Account numbers are stored as strings, so they can be entered directly from the keyboard.
- A while True menu loop keeps the program running until the user chooses to exit.
- A console interface avoids unnecessary external dependencies.
- All checks run before any balance is changed.

ASSUMPTIONS AND CONSTRAINTS :

A few simplifying assumptions were made to keep the project within the scope of a first-semester course, while still producing a usable tool.

- A single user runs the application on a single machine at a time.
- All amounts are in a single currency (Rs.), without conversion.
- The user enters account numbers and amounts through the keyboard.
- The program is run from a terminal with a standard Python 3 interpreter available.
- Data only needs to exist for the length of one session.

CHALLENGES AND SOLUTIONS :

Preventing invalid transactions was the main challenge. Users can enter negative amounts, amounts larger than the balance, or account numbers that do not exist. The solution was to validate every input before changing any data, rejecting bad input at the point of entry.

Avoiding duplicate accounts was solved by using the account number as the dictionary key and checking membership before creating a new entry.

Keeping the program readable as features were added was handled by giving each operation its own function instead of writing everything inside the menu loop.

PROJECT TIMELINE AND WORK BREAKDOWN :

The project was carried out in stages, moving from requirements to a working, tested application. The approximate breakdown of effort is shown below.

Stage	Activity
1	Requirements gathering and problem definition
2	Data structure and function design
3	Core implementation (create account, check balance)
4	Transaction logic (deposit, withdraw) and validation
5	Menu integration and manual testing
6	Test execution and result recording
7	Report documentation and screenshot collection

LEARNINGS AND OUTCOMES :

The project provides practical experience in converting a problem into a structured Python application and connecting user input, processing and output.

- Functions and program structure
- Nested dictionaries
- Loops and conditional logic
- Input validation
- Manual test design
- Project documentation

LIMITATIONS AND FUTURE ENHANCEMENTS :

Limitations

- Data is not persistent; it is lost when the program closes.
- No exception handling for non-numeric amount input.
- No authentication (PIN or password).
- Negative initial deposits are not checked at account creation.
- Floating-point numbers are used for money.

Future enhancements

- Save data to a file or database (JSON, SQLite).
- Add try/except blocks for invalid input.
- Add PIN-based login and transaction history.

VITYARTHI – PROJECT

- Use classes (OOP) and the decimal module for accuracy.
- Add transfers between accounts.
- Build a GUI or web interface.

CONCLUSION :

The Bank Management System demonstrates how an everyday problem can be addressed with a structured Python application. It combines menu-driven input handling, validation, dictionary-based data storage and function-based design to implement the core operations of a bank account system.

The current version provides a clear foundation that can be extended with persistence, security and a richer interface, while remaining suitable for an introductory academic project.

GLOSSARY OF TERMS :

Term	Meaning
Account number	A unique identifier used as the dictionary key for each account.
Balance	The amount of money currently held in an account.
Deposit	Adding money to an account.
Withdrawal	Removing money from an account.
Validation	Checking that input meets required rules before use.
Dictionary	A Python data structure that stores key-value pairs.
Persistence	Storing data so it survives after the program closes.
CLI	Command-Line Interface; a text-based way of using a program.

USER MANUAL – HOW TO RUN THE APPLICATION :

The following steps describe how to run the Bank Management System from a terminal.

- 1. Install Python 3 if it is not already available on the system.
- 2. Open a terminal and navigate to the project folder.
- 3. Run the application with: `python main.py` (or the name of your .py file).
- 4. Choose an option from the numbered menu (1 to 6).
- 5. Follow the prompts to enter account number, name, or amount.
- 6. Select option 6 to exit the program.

REFERENCES AND SUBMISSION CHECKLIST :

The implementation uses core Python concepts and course learning context. Before submission, all project details and runtime behaviour should be verified.

- Python documentation: docs.python.org
- CSE 1021 course learning materials
- Student-created implementation and test cases
- Run every menu option
- Add screenshots from your own execution
- Verify student details and date
- Check the final ZIP contents